Discriminative and Generative Neuronal Networks on FSD Break Data

Chenwei Li, Jingyao Zhang, Moritz, Xuliang Xu

June 19, 2025

1 Introduction

The coding project of the Neural Network Seminar thought by Marina Dolokov in 2025 is about processing data of break measurements collected by the company FSD in Radebeul. The project consists out of two groups tackling different aspects of deep learning in the context of the given data. One group of students working on the bias in the dataset to enhance the learning process. The second group tries to create a generative model to increase the set on which further modells can be trained. In this short overview, we present the thought, problems and results of our group working on the generative AI task although also in this group, preprocessing of the data plays a crucial role.

1.0.1 Dataset and Task

The dataset consists of multi-channel time series sensor data, stored in pickle files (`train.pickle`, `test.pickle`). Each sample includes a sequence of acceleration and rotation sensor readings in x,y,z-directions and a corresponding class label. The goal is to build a neural network model that accurately classifies the braking state. We also try to generate new data to enhance the classifiers performance.

Contents

2 Task 1: FCN, 2D CNNs vs. LSTMs Parameter Tests & Comparison

2.1 FCN: Fully Connected Network

(Jingyao Zhang)

Sensor data classification is a critical task in analyzing time-series data for various applications such as fault detection and predictive maintenance. This section documents our efforts to improve Fully Convolutional Network (FCN) models for sensor data classification. The research includes experiments with different methods, pipelines, and data preprocessing techniques to enhance model performance. Notably, we explored the use of Generative Adversarial Networks (GANs) for data augmentation, label simplifications, and strategies to avoid information leakage.

2.1.1 Model Architecture

The FCN model consists of three convolutional layers followed by a global average pooling layer and a fully connected output layer. The primary architecture includes:

- Convolutional layers with kernel sizes of 8, 5, and 3 to extract hierarchical features from sensor data.
- Adaptive average pooling for dimensionality reduction.
- Dropout layers with probabilities ranging from 0.3 to 0.5 to prevent overfitting.
- Fully connected layers for classification tasks.

Modifications such as dropout and L2 regularization were applied to further optimize the model's generalization capability.

- **Input Layer:** item Accepts input data with a shape corresponding to the flattened sensor data, i.e., $input\_size$.
- **Hidden Layers:**
  - Hidden Layer 1:
    - Fully connected layer with 128 units.
    - Activation function: ReLU.
    - Dropout layer with a dropout rate of 0.4 to reduce overfitting.
  - Hidden Layer 2:
    - Fully connected layer with 64 units.
    - Activation function: ReLU.
    - Dropout layer with a dropout rate of 0.3 to reduce overfitting.
- **Output Layer:**
  - Fully connected layer with $num\_classes$ units, where $num\_classes$ represents the number of categories to classify.

  - Activation function: Softmax, providing normalized probabilities for multi-class classification.

- **Loss Function:**
  - CrossEntropyLoss, suitable for multi-class classification problems.

- **Optimizer:**
  - Adam optimizer with a learning rate of 0.001, enabling adaptive learning for faster convergence.

2.1.2 Pipeline Workflow

The FCN pipeline workflow consists of multiple steps aimed at preprocessing, balancing, and standardizing data to make it suitable for training:

1. **Data Loading**
   - Original training data is loaded from `train.pickle`, comprising sensor data and labels.
   - GAN-generated data is loaded from `generated_data.pickle`, offering synthetic sensor data and labels.
   - Test data is loaded from `test.pickle`, used for final evaluation.
2. **Data Augmentation**
   - GAN-generated data is merged with the original training data.
3. **Class Balancing**
   - Class imbalance in the training data is addressed using `resample`, upsampling minority classes to match the maximum class count.
4. **Data Standardization**
   - Training and test data are standardized to zero mean and unit variance, ensuring consistent numerical ranges.
5. **Train-Validation Split**
   - Training data is split into training and validation sets using `train_test_split`.
6. **Label One-Hot Encoding**
   - Labels for training, validation, and test sets are transformed into one-hot encoded formats for multi-class classification.
7. **Data Reshaping**
   - Sensor data is reshaped into dense input format, suitable for feeding into a fully connected network.
8. **PyTorch Data Preparation**

- Training and validation datasets are wrapped into `TensorDataset` objects.
- DataLoaders are created with batching for efficient training.

9. **Model Definition**
   - A Fully Connected Neural Network (FCN) is defined with:
     - Three linear layers with 128, 64, and number of classes as units.
     - Dropout layers to mitigate overfitting.
     - ReLU activations and a softmax output for probabilities.
   - Loss function: Cross-Entropy Loss.
   - Optimizer: Adam.
10. **Model Training**
    - Training loop iterates over epochs, optimizing the model using backpropagation.
    - Validation is conducted at the end of each epoch to monitor performance.
11. **Model Evaluation**
    - Test accuracy is computed by evaluating the model on the test set.

2.1.3 Experiments and Evolution

We conducted several experiments to optimize the FCN performance:

1. **1Evo: Initial Attempts to Improve FCN**

   The "1Evo" experiments (see `FCN_1Evo_Methods.ipynb`) focuses on baseline exploration, model regularization, and data preprocessing for the FCN model.

   - Random Splitting and Cross-validation
   - Model Architecture and Regularization
   - Data Normalization MaxAbs Scaling and Z-score Standardization
   - Handling Class Imbalance
   - Early Stopping

2. **UseTest: Incorporating Test Data Categories**

   The "UseTest" experiment (see `FCN_UseTest_bestAcc.ipynb`) explores training the FCN model using only the classes/categories present in the test data, which leads to higher performance, but introduces information leakage.

3. **BestFCN2Evo: GAN-Augmented Training with Two Labels**

   The "BestFCN2Evo" experiment (see `BestFCN_2Evo_withOUT_fliter.ipynb`) presents the best and most robust pipeline, this pipeline avoids the use of test data and employs GANs for data augmentation. Additionally, sensor data labels were simplified

into two categories: "working" and "non-working." The GAN-augmented training strategy ensures balanced class distributions and eliminates information leakage, leading to the best model performance to date.

The table below summarizes the results obtained at each step.

Model	**Key Features**	**Best Accuracy**
1Evo	• Random split (5-fold) • Normalization and Augmentation • Dropout/L2	(train with gan) Val Acc: 0.9478 Test Acc: 0.6513
UseTest	• Test categories used for training • MaxAbs scaling • Dropout (0.3), L2 regularization	Val Acc: 0.9478 Test Acc: 0.6454
BestFCN2Evo	• GAN data augmentation • Label simplification (2 classes) • Data balancing (upsampling) • Standardization (mean-variance) • Dropout, L2, 5-fold CV	Val Acc: 0.7838 Test Acc: 0.5485

Table 1: Summary of key features and best accuracy for each model pipeline.

2.1.4 Result Analysis

- **1Evo Results** The 1Evo experiments revealed that while random splits and normalization techniques improved validation accuracy, test accuracy remained inconsistent.

- **UseTest Results** The UseTest pipeline achieved high test accuracy by leveraging test data categories during training. However, the presence of information leakage invalidates these results for real-world deployment.

- **BestFCN2Evo Results** The BestFCN2Evo pipeline produced the most reliable and robust model. By combining GAN-augmented data and simplified labels, the model achieved a test accuracy of 54.85%. This represents a significant improvement over previous approaches without compromising data integrity.

2.1.5 Conclusion and Future Work

This section demonstrates the evolution of FCN models for sensor data classification, culminating in the BestFCN2Evo pipeline. The use of GANs for data augmentation and label simplifications proved instrumental in improving model performance. Future work will focus on:

- Exploring advanced GAN architectures for better data augmentation.

- Investigating additional preprocessing techniques to enhance feature extraction.

- Expanding the model's application to multi-label classification tasks.

These advancements will further refine the pipeline and extend its utility to broader applications in sensor data analysis. subsection

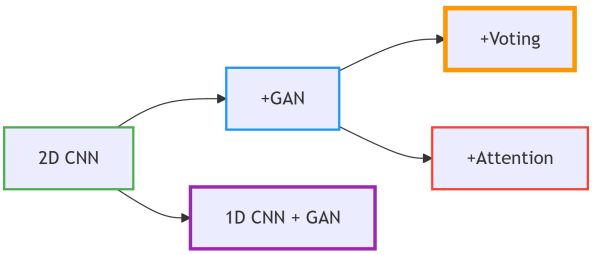


Figure 1: Model Architecture Spectrum

2.2 2D CNN

(Chenwei Li)

Convolutional Neural Networks (CNNs) exploit local connectivity and weight sharing to efficiently learn hierarchical features from grid-structured inputs. A 2D CNN applies convolutional kernels over spatial dimensions, making it well suited for extracting patterns from two-dimensional sensor representations (e.g., spectrogram-like matrices).

2.2.1 Model Variants & Rationale

1. 2D CNN (Baseline)

- **Architecture**: Treats sensor data as a "1×128×6" image. Three Conv2D→BatchNorm→ReLU→MaxPo blocks, followed by GlobalAveragePooling + Dense → Softmax.
- **Purpose**: Simple baseline for spatial feature extraction.
- **Test Accuracy**: Õ.51–0.57 (limited generalization).

2. 2D CNN + GAN

- **Addition**: Incorporates GAN-generated synthetic samples into training data.
- **Purpose**: Augment minority fault classes.
- **Test Accuracy**: Õ.57–0.64 (medium stability, sensitive to GAN quality).

3. 2D CNN + GAN + Random Split Voting (Ensemble)

- **Description**: Trains multiple 2D CNN+GAN models on different random splits; aggregates predictions via hard or soft voting.
- **Purpose**: Reduce variance and enhance robustness.
- **Test Accuracy**: Consistently ¿0.60 (best and most stable).

4. 2D CNN + GAN + Multi-Head Attention

- **Description**: Adds multi-head self-attention after convolution blocks to weight feature positions.
- **Purpose**: Focus on critical time-channel patterns.
- **Test Accuracy**: Õ.50–0.60 (unstable, often worse than baseline).
- **Failure Analysis**:

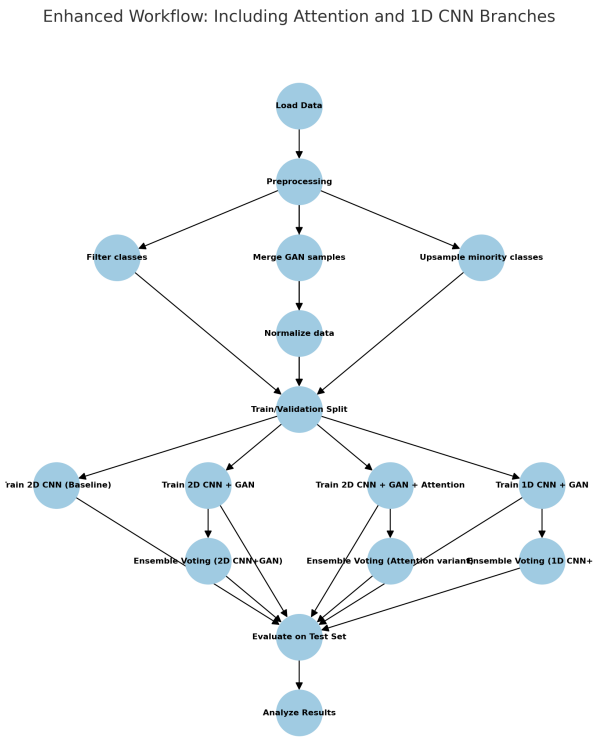


Figure 2: workflow

- Brake faults depend on local temporal spikes; global attention may mis-weight signals.
- Shallow CNN backbone limits feature abstraction for effective attention.

5. 1D CNN + GAN

- **Description**: Applies 1D convolutions along the time axis with GAN-augmented data.
- **Purpose**: Directly capture temporal patterns with fewer parameters.
- **Test Accuracy**: $\tilde{0}$.59–0.63 (stable, slightly below ensemble).

2.2.2 Workflow Overview

- **Data**: `train.pickle`, `generated_data.pickle`, `test.pickle`.
- **Preprocessing**: Filter classes, merge GAN data, upsample minority classes, normalize with training mean/std.
- **Training**: Adam optimizer, CrossEntropyLoss with class weights, ReduceLROnPlateau, EarlyStopping.
- **Evaluation**: Test accuracy, F1 scores, confusion matrices.

2.2.3 Results & Analysis

1. Performance Comparison
2. Test Accuracy Visualization

Insight: The ensemble model (2DCNN+GAN+Vote) consistently outperforms others, while the attention variant shows high variability.

Model	Test Accuracy Range	Stability	Key Observations
2D CNN + GAN + Vote	**bigger than 0.60**	Medium	Best stability & performan
1D CNN + GAN	0.59–0.63	High	Stable, outperforms 2D ba
2D CNN + GAN	0.57–0.64	Medium	Sensitive to GAN quality
2D CNN (baseline)	0.51–0.57	High	Limited generalization
2D CNN + GAN + Multi-Head Attn	0.50–0.60	Low	Attention unstable, often v

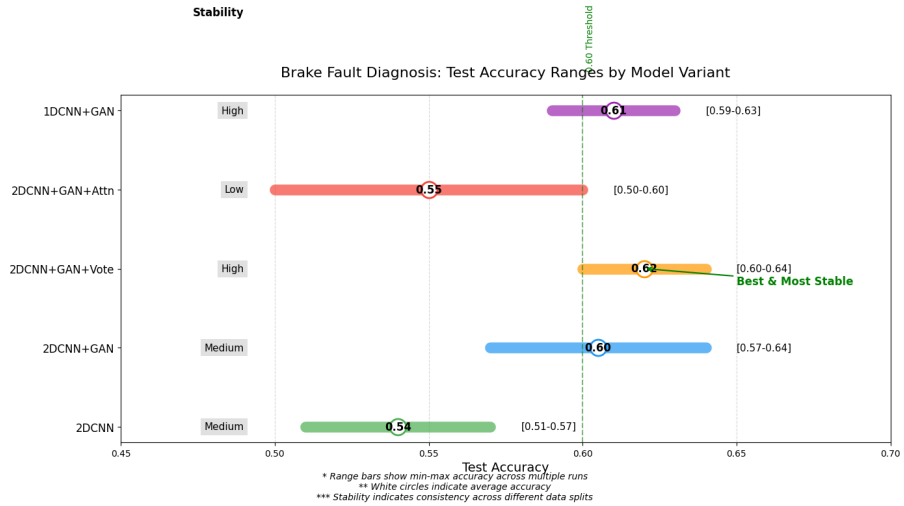


Figure 3: Test Accuracy

2.2.4 Core Issues & Diagnosis

- **Class Imbalance**: Minority classes (e.g., 1,2,4,5) have recall ¡0.35; "no-fault" class recall 0.96, skewing predictions.
- **Attention Failure**: Global attention mis-weights local temporal spikes critical to brake faults; shallow CNN limits abstraction.
- **Overfitting & Shift**: Validation accuracy ¿95% vs. test õ60%; train/validation embeddings diverge from test data.

2.2.5 Conclusions & Future Work

1. Core Conclusions

- **Ensemble Success**: 2D CNN + GAN + Voting achieves ¿0.60 test accuracy with high stability.
- **Attention Ineffective**: Multi-head attention degrades performance in shallow CNNs.
- **Data Quality Key**: GAN samples must align with real distributions.
- **1D CNN Viable**: Stable at õ0.59–0.63, suits temporal data.

2. Future Work

- Optimize GANs with physical constraints and domain-adversarial training.
- Explore hybrid 1D+2D models or lightweight attention (e.g., SE blocks).

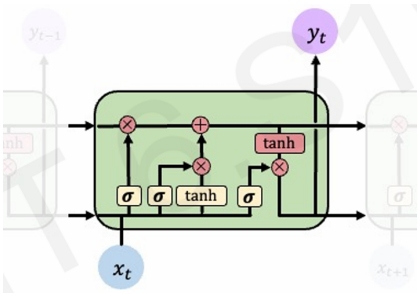
- Apply advanced augmentation (time-warping, noise injection).
- Use stratified k-fold CV and monitor per-class metrics.
- Enhance ensembles with diverse architectures.

2.3 LSTM: Long Short-Term Memory Model

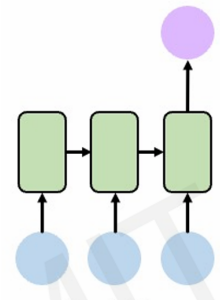
(Moritz Bauer)

2.3.1 Model Architecture

We recall the scheme of the lecture of LSTM networks.



(a) A single LSTM-Cell



(b) Multiple Iterations of the Dataset to get a classification

Since we use the acceleration and velocity measurements of one car with corresponding dimension $(128, 6)$, we have 128 LSTM-cells expecting 6 data points each. Every cell is predicting a break condition code but only the last prediction is taken into account. The model is implemented in PyTorch. Further specifications:

- Hidden LSTM layer is fully connected to output layer
- Random-Split Validation of test and validation data is renewed every epoch
- Training data is shuffled before training
- use of Adam Optimizer
- use of Binary Cross Entropy Loss-Function

2.3.2 Pipeline Workflow

We used quite some time to improve the discriminative models, since the results haven't been too satisfying. The outlines of the process for the LSTM model are given below:

1. Label conversion and normalization of data
   - We simplified the label to train our model with, we only differ by working/defect breaks by using the map for label x

$$x \mapsto \begin{cases} 1 \; if \; x = 0 \\ 0 \; if \; x > 0 \end{cases} \quad \text{later:} \quad x \mapsto \begin{cases} 1 \; if \; x > 0 \\ 0 \; if \; x = 0 \end{cases}$$

   - And for the purpose of easier detection, we normalized the data for the velocity and rotation measurement each by

$$AX_{norm} = (\tfrac{ax_1}{ax_{max}}, ..., \tfrac{ax_{128}}{ax_{max}}), \; ax_{max} = max\{|a_1|, ..., |a_{128}|\}$$

so we do not shift the data but ensure the maximal absolute value to be 1.
In fact, both methods turn out to hinder the training process. Especially our label conversion caused antilearning (significant bad results), since we have

$$\begin{aligned} &\text{label } x = 0 \text{ (break defect)} && \rightarrow \text{LSTM needs to detect irregularity} \\ &\text{label } x = 1 \text{ (break in order)} && \rightarrow \text{LSTM needs to detect regularity} \end{aligned}$$

but somehow the model expects to detect any "property" for label $x = 1$. To solve this, we turned around the images of the map.

2. Bias in the Data

   - Since our training data exists out of around 40% working breaks and 60% defect breaks, our model often learns to predict "This is a broken break" since it succeeds in 60% of the cases.
   - To prevent this, we use weights corresponding to the label distribution on the loss function to punish our model harder if it falsely predicts a working break

3. Hard coded data extension

   - Since our test dataset only exists out of around 4000 measurements with varying models and break conditions, the amount of examples the model has per case (e.g. model & break condition) is limited.
   - To extend the data, we shifted data in time and approximated up to 5 new time steps or scaled the data by small factors of 0.95-1.05. This is to force the model to focus on the relevant information into the data.

We compare the development of the accuracy from step 1-3 under the following conditions:

Number and Dimension of Hidden Layer	1 x 512
Batch Size	32
Number and Size of Epochs	3 x 20
Ratio Test/Validation Data	3/1
Learning Rate	10^{-3}

Setting	**Using 1**	**Not Using 1**	**Using 2**	**Using 2&3**
Total accuracy	0.4	0.52	0.56	0.51
Working break accuracy	0	0.49	0.84	0.61
Broken break accuracy	1	0.67	0.36	0.36

At first place, hard coded data extension does not increase our models' performance. If we plot the loss over the training process, the training seems to be quite unstable. To stabilize our training process, we now try to gather 64 measurements in one batch. We keep our weighted loss function and test different initial learning rates for the Adam optimizer:

Setting	$lr = 10^{-1}$, $b = 64$	$lr = 10^{-2}$, $b = 64$	$lr = 10^{-4}$, $b = 64$	$lr = 10^{-5}$, $b = 64$
Total accuracy	0.68	0.68	0.57	0.49
Working break accuracy	0.94	0.93	0.57	0.25
Broken break accuracy	0.3	0.33	0.59	0.84

2.3.3 Task 2.1

Since an initial learning rate of $10^{-1}/10^{-2}$ seemed to be the best option, we now try to train the model with learningrate $lr = 5 \cdot 10^{-2}$, augmented datasets (generated/hard coded), weights, and without normalization of the sensor data:

Dataset	**Hard Coded**	**GAN**	**RNN**	**CGAN**	**TGAN**	**DANN**
Total accuracy	0.69	0.7	0.68	0.64	0.69	0.67
Working break accuracy	0.87	0.89	0.83	0.89	0.95	0.87
Broken break accuracy	0.43	0.43	0.47	0.28	0.30	0.38

2.3.4 Result Analysis

The interpretation of the observed behaviour could be, that by normalizing the data, we mostly lose information instead of helping the model to increase its prediction. Setting weights is crucial to work against the biased training data and data extensions seems really helpful under the condition of a well chosen hyperparameter setting.
It seemed to be essential to give a sufficient large step-width to the Adam optimizer to let him "stray around" a bit to find good minima to approach against. Also observed was, that increasing the layers size and the models' depth does not necessarily increase the models' performance on the test data and only leads to longer coffee breaks.

3 Task 2: Generative AI

(Xuliang Xu)

Our group focused on Generative AI tasks. We initially implemented basic data generation using GAN and RNN models. To address training set label imbalance and distribution mismatches between training and test sets, we subsequently developed cGAN and DANN models. Finally, to capture more complex temporal patterns, we implemented transformer-based GANs.

3.1 GAN

Generative Adversarial Networks (GANs) are unsupervised learning methods involving two competing neural networks. The generator takes random latent space samples as input and produces outputs that mimic real training samples. The discriminator receives either real samples or generator outputs, aiming to distinguish between them. Through adversarial training, both networks collaborate to generate indistinguishable synthetic data.

3.1.1 Model Architecture

Both generator and discriminator are implemented in PyTorch with the following structure:

- Fully Connected Neural Networks (FCNN) serve as the base architecture
- Generator uses BatchNorm1d for stability with ReLU activation
- Discriminator employs Dropout regularization with LeakyReLU activation

3.1.2 Experiments and Evolution

Initially, the generator and discriminator had similar architectures, but the discriminator converged faster to a loss of 0.01. To achieve balanced adversarial training, we implemented:

- Increased generator capacity with more neurons
- Applied 5× higher learning rate for the generator
- Provided additional training iterations for the generator
- Used soft labels (0.9/0.1) instead of hard labels (1/0) for smoother learning

These modifications stabilized discriminator loss around 0.7, indicating optimal equilibrium where the discriminator cannot reliably distinguish real from synthetic data. The soft labeling technique proved most impactful among these adjustments.

3.1.3 Pipeline Workflow

For Task 2, we employed a distinct preprocessing approach: converting original labels into four binary sub-labels represented by multi-head outputs, enabling the model to extract more generalizable features from the data.

3.2 RNN

Recurrent Neural Networks (RNNs) are well-suited for sequential data processing, making them ideal for simulating sensor time series data. Unlike fully connected networks, RNNs employ recurrent connections where neuron outputs from one time step serve as inputs for the next, enabling the capture of temporal dependencies and sequential patterns.

3.2.1 Model Architecture

- Input: sensor data from previous layer and 4 sub-labels
- Hidden layers: 5 layers with 128-dimensional hidden states
- Sequence length: 127 time steps

3.2.2 Experiments and Evolution

During training, the RNN loss consistently plateaued around 0.14 without meaningful reduction. Despite experimenting with various configurations—different hidden layer counts, dimensions, and sequence lengths—the training loss remained stable at approximately 0.14, indicating potential convergence limitations.

3.3 cGAN

Conditional Generative Adversarial Networks (cGANs) extend traditional GANs by incorporating conditional information into both generator and discriminator networks. Unlike standard GANs that generate samples from random noise, cGANs accept both latent vectors and conditional labels as input, enabling controlled generation of specific data types.

3.3.1 Model Architecture

- Implemented using Fully Connected Neural Networks (FCNN)
- Generator employs LeakyReLU activation (unlike vanilla GAN's ReLU)

3.3.2 Experiments and Evolution

Initial cGAN training exhibited significant instability. Unlike vanilla GANs, the discriminator loss oscillated wildly between 0.01 and 100, rather than converging to small values. To stabilize training and maintain discriminator loss near 0.7, we implemented additional techniques beyond standard GAN methods:

- Switched generator activation to LeakyReLU
- Increased dropout rates in the discriminator for enhanced regularization

3.4 Transformer-GAN

Transformers are deep learning architectures utilizing attention mechanisms. To capture more complex temporal relationships and cross-sensor patterns, we integrated Transformer architecture into GAN frameworks, investigating whether this approach could achieve superior simulation results.

3.4.1 Model Architecture

- Multi-sensor processing using Transformer encoder with 6 attention heads and 3 layers
- Learnable positional embeddings to capture temporal relationships in 128-length sequences
- Flattened feature representations for downstream tasks or network component integration

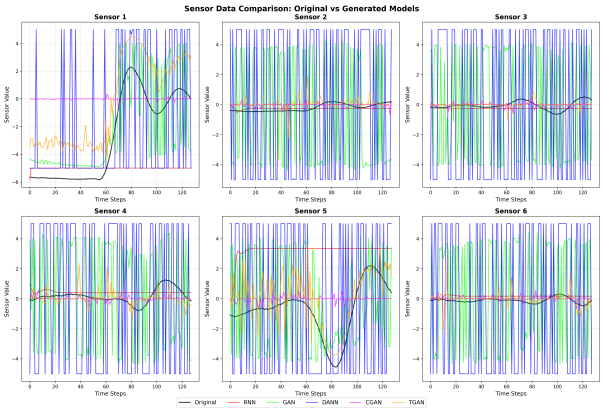


Figure 5: 6 Data Comparison

3.4.2 Experiments and Evolution

Leveraging experience from previous GAN implementations, we successfully stabilized the model training. However, the Transformer architecture demanded significantly higher computational resources, requiring over double the GPU memory and substantially longer training times compared to previous methods.

3.5 DANN

We observed that models absent from the training set appeared in the test set, reflecting real-world scenarios. To address this domain shift challenge, we employed Domain-Adversarial Neural Networks (DANN), which share architectural similarities with GANs but target domain adaptation problems. DANN uses a Label Predictor to classify features from a feature extractor, while a Domain Discriminator determines whether features originate from source or target domains, enabling extraction of domain-invariant representations.

3.5.1 Model Architecture

- Three-component architecture: FCNN Feature Extractor (1024→768→512 dimensions), Label Predictor, and Domain Classifier with adjustable gradient reversal
- Domain-Invariant Generator accepts random noise and domain-invariant features to simultaneously generate sensor data and corresponding labels
- Domain-Invariant Discriminator evaluates authenticity of generated sensor-label pairs using concatenated sensor and label information

3.6 Result Analysis

Table 2 presents the statistical comparison of mean and standard deviation values across all sensors for the original data and five different generative models: RNN, GAN, DANN, CGAN, and TGAN. The comparative analysis of the five generative models reveals distinct behavioral patterns and capabilities in sensor data simulation, as illustrated in Figure 5.

3.6.1 RNN Performance

The RNN model demonstrates remarkable stability across all sensors, evidenced by consistently low standard deviation values ranging from 0.0313 to 0.8506. As observed in the time series

Table 2: Statistical Comparison of Sensor Data: Mean and Standard Deviation

2*Sensor	Original		RNN		GAN		DANN		CGAN		TGAN	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
1	-2.7438	3.0695	-4.9706	0.0767	-0.7027	4.9493	-1.4732	4.7612	-2.1119	4.5133	-0.2270	3.1686
2	-0.2109	0.2403	0.0002	0.0658	0.0784	4.9990	0.1569	4.9919	0.3907	4.9793	0.0274	0.1564
3	-0.0520	0.2784	0.1636	0.0313	-1.0075	4.8895	-0.3650	4.9664	0.2322	4.9881	0.0027	0.0916
4	0.1585	0.4566	-1.1173	0.8506	0.1560	4.9971	0.1022	4.9779	-0.0615	4.9706	-0.0249	0.5665
5	-0.7354	1.6998	-0.8576	0.3230	0.4601	4.9685	-0.0771	4.9958	-0.3284	4.9715	0.0238	1.7119
6	-0.1126	0.1659	2.0848	0.4413	-0.0002	4.9991	-0.1579	4.9933	0.5672	4.9494	-0.0048	0.1189

plots, the RNN-generated data exhibits minimal fluctuation and fails to capture the complex temporal relationships present in the original sensor data. The model appears to converge to relatively constant values, indicating its inability to learn and reproduce the intricate patterns and dynamics inherent in the sensor measurements.

3.6.2 GAN-based Models (GAN, DANN, CGAN)

The GAN-based approaches, including vanilla GAN, DANN, and CGAN, demonstrate a significantly different behavior profile. These models exhibit high standard deviation values, consistently around 4.9 across all sensors, indicating their capability to generate data with substantial variability. This high variance suggests that these models can capture more complex patterns compared to RNN. The generated data shows erratic jumping between extreme values rather than following coherent temporal trends. While this variability indicates the models' potential to capture complex relationships, the lack of temporal coherence limits their practical applicability for sensor data simulation.

3.6.3 Transformer-based Model (TGAN)

The TGAN model presents the most promising results in terms of trend capture and temporal coherence. Unlike the GAN-based models, TGAN successfully captures the underlying trends present in the original sensor data, as evidenced by its standard deviation values that closely match those of the original data across multiple sensors. While the generated time series exhibit considerable oscillations and fluctuations during the trend evolution, the overall directional patterns and temporal dependencies are preserved. This capability to maintain trend coherence while incorporating natural variability makes TGAN the most effective model for sensor data generation among the tested approaches.

3.6.4 Computational Efficiency Considerations

Despite its superior performance in trend capture, TGAN comes with significant computational overhead. The model requires approximately four times the memory consumption compared to other models and exhibits substantially longer training and inference times.

3.7 Dataset Expansion and Performance Improvement

The analysis of performance improvement resulting from dataset expansion with synthetic data is already included in each individual model's section.

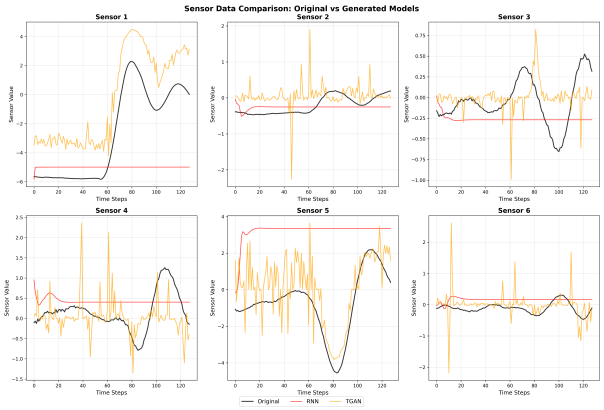


Figure 6: 3 Data Comparison

4 Comparison of different models

(Jingyao Zhang)

4.0.1 Task and Data

In the first task of the FSD project, we compare different models of neural networks to obtain a good discriminative model to predicts whether a break does work or is defect given acceleration measurements and angular velocity in three dimensions and also the cars specifications as model, weight, velocity or deceleration average. Our test and training data is labelled with a "break condition code" from 0 - 11 which we'll use in this or another form to train and test our neural network against.

4.0.2 Performance Summary

Based on our extensive experiments on the FSD brake dataset, we summarize the main findings for the three model families as follows:

Model	**Best Test Accuracy**	**Stability**	**Key Strengths/Weaknesses**
FCN (BestFCN2Evo)	0.55	Medium	Simple, robust, but limited for sequential data
2D CNN + GAN + Ensemble	>0.60	High	Best overall; ensemble/voting stabilizes results
1D CNN + GAN	0.59–0.63	High	Stable, effective for temporal features
2D CNN (baseline)	0.51–0.57	High	Limited generalization
LSTM (best)	0.68–0.70	Moderate	Good for regular temporal patterns; sensitive to bias and hyperparameters

Table 3: Summary comparison of discriminative models on FSD dataset

4.0.3 Key Observations

- **FCN**: The fully connected network serves as a solid baseline. Its performance improves with GAN-based data augmentation, regularization, and label simplification. However, it fails to exploit the temporal structure inherent in time-series sensor data, which limits its ultimate accuracy compared to more specialized models.
- **CNN**: Convolutional models, especially with GAN-augmented data and ensemble voting, consistently outperform FCNs. 2D CNNs effectively capture spatial and local temporal relationships, and ensemble methods (random split voting) greatly improve stability and generalization. 1D CNNs also show high performance, particularly for sequence-like sensor data, but slightly lag behind the best 2D CNN ensembles.
- **LSTM**: LSTM recurrent networks are well-suited for sequential dependencies in the sensor data. With optimal hyperparameters and data balancing, LSTM models can reach high accuracy (up to 0.70 in certain binary settings). However, they are sensitive to label bias, normalization, and require careful tuning. LSTMs benefit from larger batch sizes and higher learning rates for the Adam optimizer, but too much model complexity leads to diminishing returns.
- **Data Augmentation and GANs**: Across all models, augmenting the dataset with synthetic samples generated by GANs effectively improves performance, especially for minority classes. However, the quality of GAN samples is crucial—poorly aligned synthetic data can harm model generalization.
- **Class Imbalance**: All models suffer from strong class imbalance, with the majority "no-fault" class dominating predictions. Applying upsampling, class weighting, and advanced augmentation is essential for meaningful fault detection.

4.0.4 Overall Recommendation

For this brake sensor dataset, While the LSTM model achieves the highest peak test accuracy (0.68–0.70), its results are less stable and more sensitive to hyperparameters and data preprocessing. In contrast, the 2D CNN + GAN + Ensemble approach, although slightly lower in peak accuracy (¿0.60), offers more consistent performance across different runs and better balances robustness and generalizability, making it preferable for practical deployment scenarios. FCNs can serve as efficient baselines or for tasks with limited temporal context.

For future work, further improvements are expected by:

- Integrating domain knowledge into GAN training and augmentation.
- Exploring hybrid architectures (e.g., CNN-LSTM or attention-augmented CNNs).
- Applying advanced ensemble strategies and model calibration.
- Systematic monitoring of per-class recall and precision to address imbalanced learning.

4.0.5 Conclusion

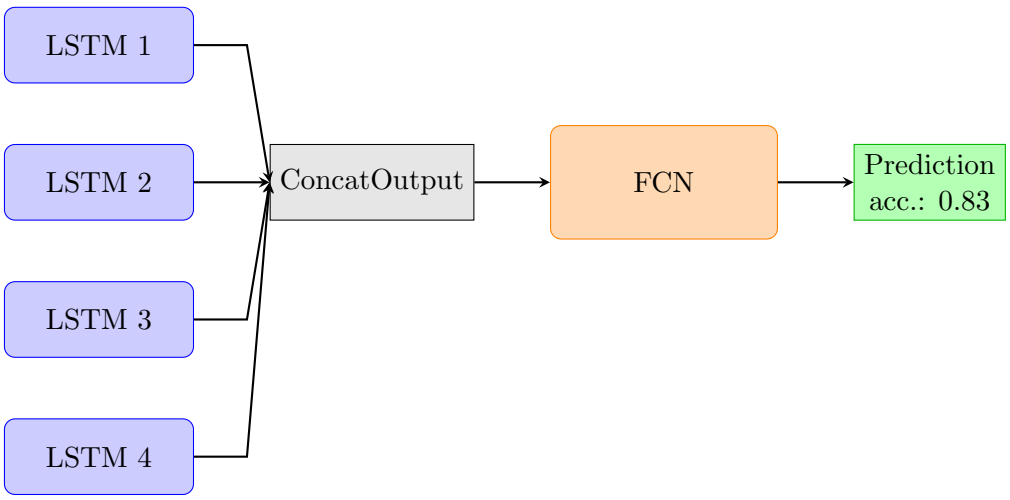
In summary, while all three model types have their merits, convolutional models with ensemble aggregation stand out for this application, mostly due to their ability to combine local feature extraction, robustness to input variability, and effectiveness with augmented data. LSTMs provide valuable insights for sequential pattern detection, but require more careful handling of data bias and hyperparameters. FCNs remain a pragmatic starting point and a performance benchmark for more complex architectures.

5 Additional Task 2: Merging Models

(Moritz Bauer)

We take 4 LSTM Networks, namely the model on the pure data with learning rate $lr = 10^{-2}$, and the models trained on hard coded data augmentation, GAN data and TGAN data.

We merged all predictions (as floats $p \in [0, 1]$) into on tensor of 4 predictions and pass it to a fully connected neural network with hidden dimension 32 to get a "merged" prediction.



This merged model is only classifying if a break is broken or not, but due to four well-trained models, we now can get a good prediction if a break is broken or not. We close this projects report with the confusion matrix of the merged model:

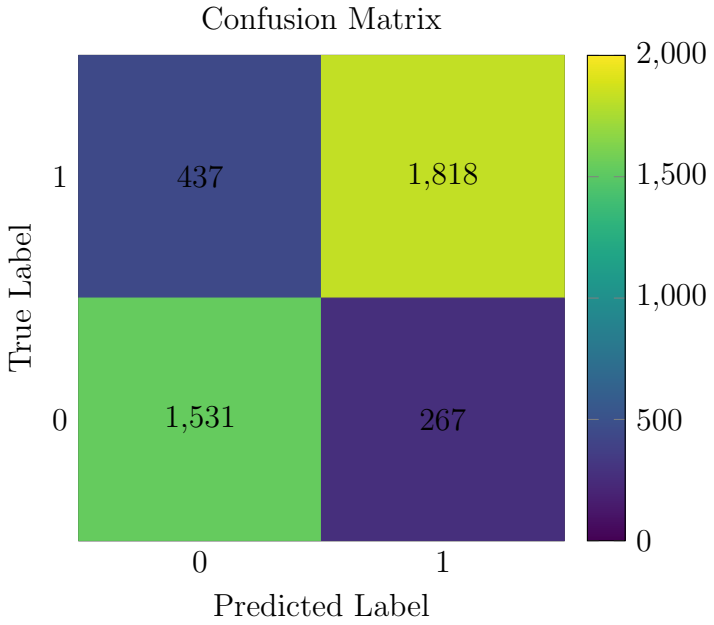


Figure 7: Right/False predictions per class 0/1 for working/broken breaks